SHIFA-ULLAH

FA24-BCS-146

- **Class:** BCS-4B
- **Lab Task:** 15-06-26
- **Implementation of doubly linked List**
- **Subject:** Data Structure

Question #1.

Write a Java program to implement a **Doubly linked list** using a **menu-driven approach**. The program should allow the user to perform the following operations:

1. **Create a doubly linked list:** Initialize an empty doubly linked list.
2. **Add a node at the beginning of the doubly linked list:** Insert a new node at the start of the list.
3. **Add a node at the end of the doubly linked list:** Insert a new node at the end of the list.
4. **Add a new node before a specified node in the doubly linked list:** Insert a new node before a given node (identified by its value).
5. **Add a new node after a specified node in the doubly linked list:** Insert a new node after a given node (identified by its value).

Program Requirements:

- Implement a class **DoublyLinkedList** with methods to perform each of the operations listed above.

- In the main program, provide a menu-driven interface that continuously prompts the user to choose an operation until they choose to **exit**.
- Validate inputs where necessary, for instance, when adding nodes before or after a specified node.
- Display the list after each modification to provide feedback to the user.

Additional Constraints (Optional):

- Ensure the linked list handles edge cases such as inserting into an empty list, inserting at the beginning when the list has nodes, and adding nodes before or after nodes that are not present in the list.

PROGRAM :-

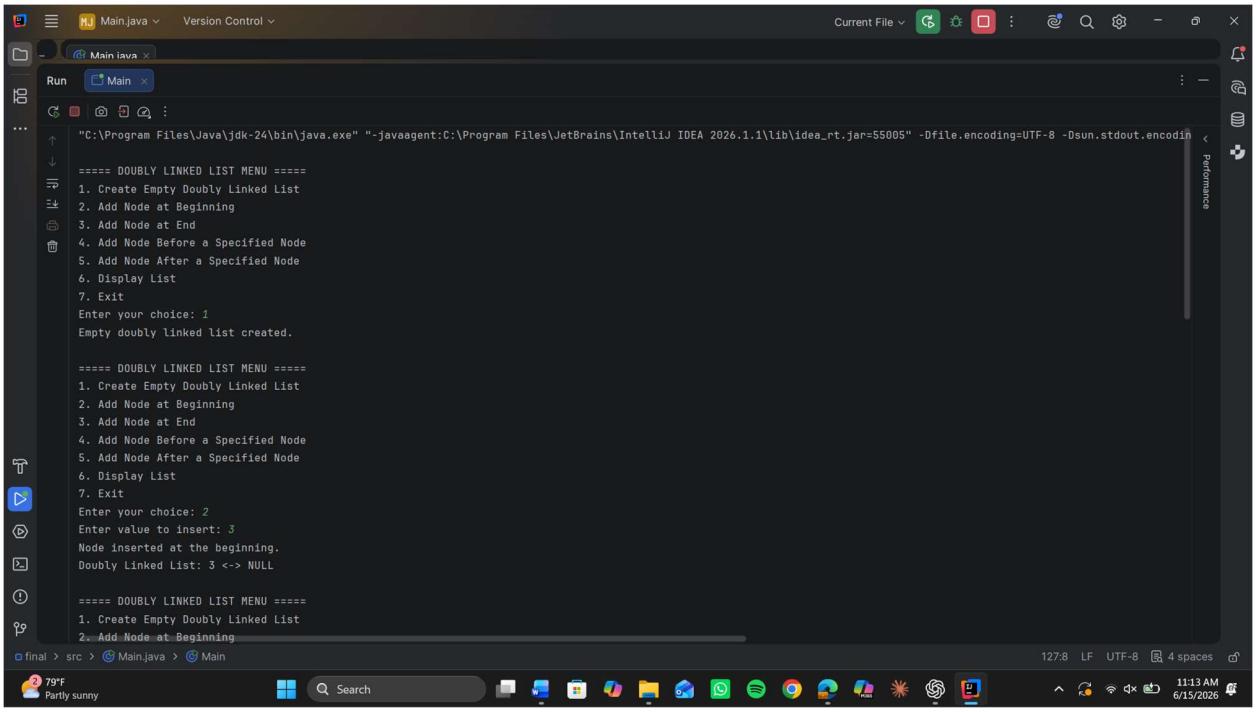
```
import java.util.Scanner;
class DoublyLinkedList
{
    class Node
    {
        int data;
        Node prev, next;
        Node(int data)
        {
            this.data = data;
            prev = null;
            next = null;
        }
    }
    Node head;
    public DoublyLinkedList()
    {
        head = null;
    }
    public void addAtBeginning(int data)
    {
        Node newNode = new Node(data);
        if (head == null)
        {
            head = newNode;
        } else
        {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
        System.out.println("Node inserted at the beginning.");
```

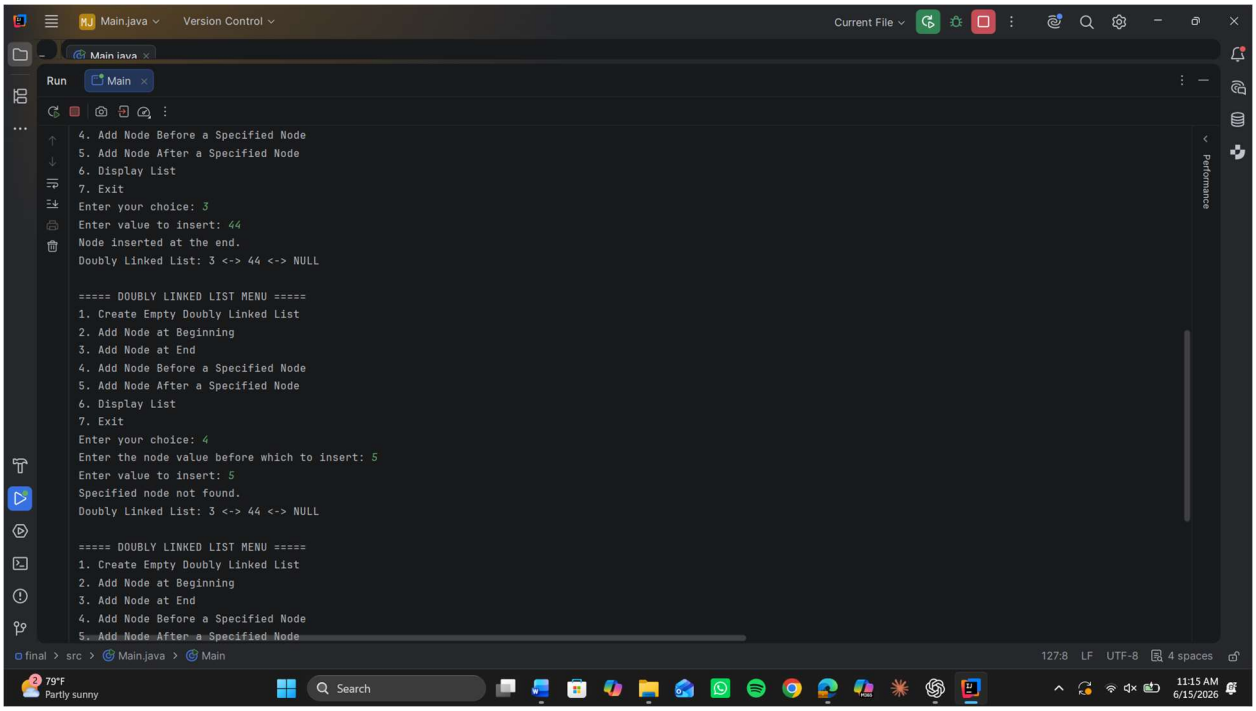
```
    }
    public void addAtEnd(int data)
    {
        Node newNode = new Node(data);
        if (head == null)
        {
            head = newNode;
        } else
        {
            Node temp = head;
            while (temp.next != null)
            {
                temp = temp.next;
            }
            temp.next = newNode;
            newNode.prev = temp;
        }
        System.out.println("Node inserted at the end.");
    }
    public void addBeforeNode(int target, int data)
    {
        if (head == null)
        {
            System.out.println("List is empty.");
            return;
        }
        Node temp = head;
        while (temp != null && temp.data != target)
        {
            temp = temp.next;
        }
        if (temp == null)
        {
            System.out.println("Specified node not found.");
            return;
        }
        Node newNode = new Node(data);
        if (temp == head)
        {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        } else
        {
            newNode.prev = temp.prev;
            newNode.next = temp;
            temp.prev.next = newNode;
            temp.prev = newNode;
        }
        System.out.println("Node inserted before " + target + ".");
    }
    public void addAfterNode(int target, int data)
    {
        if (head == null) {
            System.out.println("List is empty.");
            return;
        }
```

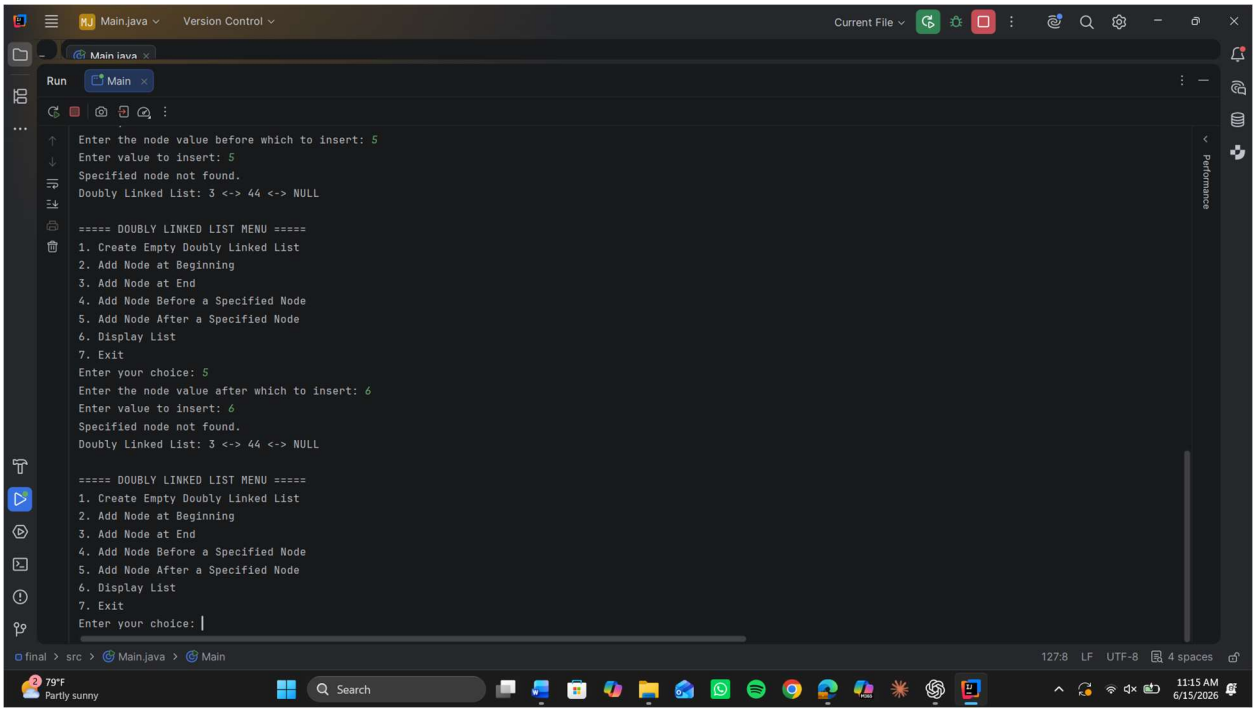
```
        Node temp = head;
        while (temp != null && temp.data != target)
        {
            temp = temp.next;
        }
        if (temp == null)
        {
            System.out.println("Specified node not found.");
            return;
        }
        Node newNode = new Node(data);
        newNode.next = temp.next;
        newNode.prev = temp;
        if (temp.next != null)
        {
            temp.next.prev = newNode;
        }
        temp.next = newNode;
        System.out.println("Node inserted after " + target + ".");
    }
    public void display()
    {
        if (head == null)
        {
            System.out.println("List is empty.");
            return;
        }
        Node temp = head;
        System.out.print("Doubly Linked List: ");
        while (temp != null)
        {
            System.out.print(temp.data + " <-> ");
            temp = temp.next;
        }
        System.out.println("NULL");
    }
}
public class Main
{
    public static void main(String[] args)
    {
        Scanner input = new Scanner(System.in);
        DoublyLinkedList list = new DoublyLinkedList();
        int choice;
        do
        {
            System.out.println("\n===== DOUBLY LINKED LIST MENU =====");
            System.out.println("1. Create Empty Doubly Linked List");
            System.out.println("2. Add Node at Beginning");
            System.out.println("3. Add Node at End");
            System.out.println("4. Add Node Before a Specified Node");
            System.out.println("5. Add Node After a Specified Node");
            System.out.println("6. Display List");
            System.out.println("7. Exit");
            System.out.print("Enter your choice: ");
            choice = input.nextInt();
            switch (choice)
```

```
            {
                case 1:
                    list = new DoublyLinkedList();
                    System.out.println("Empty doubly linked list created.");
                    break;
                case 2:
                    System.out.print("Enter value to insert: ");
                    int beginData = input.nextInt();
                    list.addAtBeginning(beginData);
                    list.display();
                    break;
                case 3:
                    System.out.print("Enter value to insert: ");
                    int endData = input.nextInt();
                    list.addAtEnd(endData);
                    list.display();
                    break;
                case 4:
                    System.out.print("Enter the node value before which to
insert: ");
                    int targetBefore = input.nextInt();
                    System.out.print("Enter value to insert: ");
                    int beforeData = input.nextInt();
                    list.addBeforeNode(targetBefore, beforeData);
                    list.display();
                    break;
                case 5:
                    System.out.print("Enter the node value after which to
insert: ");
                    int targetAfter = input.nextInt();
                    System.out.print("Enter value to insert: ");
                    int afterData = input.nextInt();
                    list.addAfterNode(targetAfter, afterData);
                    list.display();
                    break;
                case 6:
                    list.display();
                    break;
                case 7:
                    System.out.println("Exiting program...");
                    break;
                default:
                    System.out.println("Invalid choice. Please try again.");
            }
        } while (choice != 7);
        input.close();
    }
}
```

OUTPUT :-


Main.java
Version Control
Current File
Run
Main
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=55005" -Dfile.encoding=UTF-8 -Dsun.stdout.encodin
===== DOUBLY LINKED LIST MENU =====
1. Create Empty Doubly Linked List
2. Add Node at Beginning
3. Add Node at End
4. Add Node Before a Specified Node
5. Add Node After a Specified Node
6. Display List
7. Exit
Enter your choice: 1
Empty doubly linked list created.
===== DOUBLY LINKED LIST MENU =====
1. Create Empty Doubly Linked List
2. Add Node at Beginning
3. Add Node at End
4. Add Node Before a Specified Node
5. Add Node After a Specified Node
6. Display List
7. Exit
Enter your choice: 2
Enter value to insert: 3
Node inserted at the beginning.
Doubly Linked List: 3 <-> NULL
===== DOUBLY LINKED LIST MENU =====
1. Create Empty Doubly Linked List
2. Add Node at Beginning
final > src > Main.java > Main
127:8 LF UTF-8 4 spaces
79°F
Partly sunny
Search
11:13 AM
6/15/2026


Main.java
Version Control
Current File
Run
Main
4. Add Node Before a Specified Node
5. Add Node After a Specified Node
6. Display List
7. Exit
Enter your choice: 3
Enter value to insert: 44
Node inserted at the end.
Doubly Linked List: 3 <-> 44 <-> NULL
===== DOUBLY LINKED LIST MENU =====
1. Create Empty Doubly Linked List
2. Add Node at Beginning
3. Add Node at End
4. Add Node Before a Specified Node
5. Add Node After a Specified Node
6. Display List
7. Exit
Enter your choice: 4
Enter the node value before which to insert: 5
Enter value to insert: 5
Specified node not found.
Doubly Linked List: 3 <-> 44 <-> NULL
===== DOUBLY LINKED LIST MENU =====
1. Create Empty Doubly Linked List
2. Add Node at Beginning
3. Add Node at End
4. Add Node Before a Specified Node
5. Add Node After a Specified Node
final > src > Main.java > Main
127:8 LF UTF-8 4 spaces
79°F
Partly sunny
Search
11:15 AM
6/15/2026


Main.java
Version Control
Current File
Main.java
Run
Main
Enter the node value before which to insert: 5
Enter value to insert: 5
Specified node not found.
Doubly Linked List: 3 <-> 44 <-> NULL
===== DOUBLY LINKED LIST MENU =====
1. Create Empty Doubly Linked List
2. Add Node at Beginning
3. Add Node at End
4. Add Node Before a Specified Node
5. Add Node After a Specified Node
6. Display List
7. Exit
Enter your choice: 5
Enter the node value after which to insert: 6
Enter value to insert: 6
Specified node not found.
Doubly Linked List: 3 <-> 44 <-> NULL
===== DOUBLY LINKED LIST MENU =====
1. Create Empty Doubly Linked List
2. Add Node at Beginning
3. Add Node at End
4. Add Node Before a Specified Node
5. Add Node After a Specified Node
6. Display List
7. Exit
Enter your choice:
Performance
final > src > Main.java > Main
127:8 LF UTF-8 4 spaces
79°F
Partly sunny
Search
11:15 AM
6/15/2026